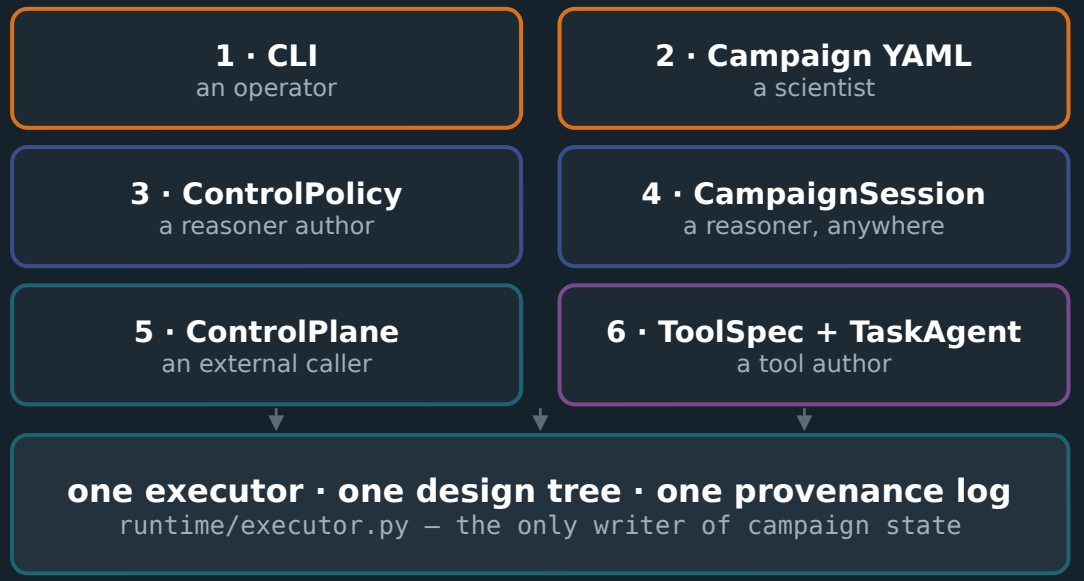
Six APIs, one engine

A policy, a tool and an external caller write against three different surfaces — and are admitted through one set of gates.

11 tools in **5 toolkits** · **10 session operations** · **19 control-plane operations** · **168 tests**

Derived against slides/impress-a-api-user-slides @ bf32d1d · every figure mined by slides/api/run_model.py, except the live runs on 21-22, which are transcribed and say so

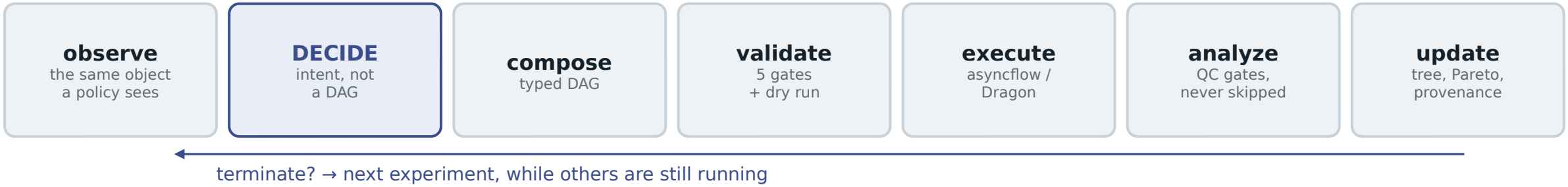
WHO WRITES AGAINST WHAT



and one place that refuses them: five gates, a dry run, and a trust interlock

ORIENTATION

What this deck shows, and what it does not



What ran, for real, to make this deck

- `mock-stabilize.yaml --model D` end to end in 2.8 s: 3 graphs composed, 1 refused, 2 admitted
- 4 design nodes absorbed, Pareto front of 3, stop reason "target front size 3 reached"
- 14 provenance records across 5 kinds, 7 ledger records, 2 pattern signatures
- 168 tests, 9 files, all of it on a laptop with no allocation

What that does not demonstrate

- Every stage was a MOCK. No GPU, no PyRosetta, no Boltz — the science is stubbed and the numbers are drawn, not computed
- One experiment in flight at a time. There is no concurrent run in this deck and no serial baseline, so nothing here divides into a speedup
- On the real six-stage chain, `replicas > 1` has never executed (backlog A10). It runs here, at width 2, against mocks
- Nothing outside this package has ever driven a campaign. The control plane is tested, not used

mined: slides/api/run.json · the run is reproducible: python slides/api/run_model.py

HOW TO LISTEN TO THIS

Three questions, and the one I cannot answer

FUNCTIONALITY	PERFORMANCE	USABILITY
What can you write against this that you cannot write against a pipeline?	*What does the loop cost, and where does it serialise?*	*Is it an application, a platform, or a component?*
a tool is 15 declared fields, not a code path	2.8 s end to end, 14 provenance records	application: one YAML, one command
a reasoner is 3 methods, or one	the executor is the sole writer of state	platform: 11 tools discovered at runtime
5 QC gates, 20 fixtures, never skippable	one experiment in flight · no baseline	component: nothing has ever driven it

The weak one is usability, and specifically the component claim. The control plane and the remote session are the two surfaces in here designed with the most care — request/response, nothing live crossing the wire, a long poll so a reasoner on a login node blocks exactly as it would in-process — and no caller outside this package has ever used either. **That is the first thing I would want to break if I were you.**

the dividers repeat this per act, with each act's weakest claim before the act

ORIENTATION

The loop is ours. Who drives it is the pluggable part.

```
observe -> DECIDE -> compose -> validate -> execute -> analyze -> update -> terminate?
```

What changed, and why it matters to an API

- The manager used to BE the loop: it called `decide()` once per cycle and blocked on the graph it asked for, so the reasoner could not be consulted while work was running.
- Now the reasoner is a coroutine that calls the campaign.

What did not change

- Every step above still happens in that order, for any one experiment.
- Models A-D are untouched: `SequentialPolicyDriver` makes a `decide`-style policy look like a self-driving one.

decide(obs) -> Decision
answer one question at a time · auditable · models A-D

`policy/base.py:19`

conduct(session)
drive yourself · submit many, collect as they finish

`policy/driver.py:35`

CampaignExecutor — owns all campaign state, reaps runs as they finish, decides termination
budget, stagnation and repeated failure are facts about state a reasoner cannot see, so stopping is not the reasoner's to declare

manager.py:82 — whichever of the two coroutines finishes first ends the campaign

THE MAP FOR THE REST OF THE DECK

Six surfaces, six audiences

Surface	Who writes against it	What they write	Size	Where
1 · CLI	an operator	3 subcommands, 6 control models	`run / tools / preflight`	`cli/__init__.py:31`
2 · Campaign YAML	a scientist	the scientific question: stages, params, objectives	`22 fields`	`runtime/executor.py:58`
3 · ControlPolicy	a reasoner author	decide() or conduct(), returning abstract intent	`3 methods · 4 decisions`	`policy/base.py:19`
4 · CampaignSession	a reasoner, in or out of process	calls onto a running campaign	`10 operations`	`core/session.py:56`
5 · ControlPlane	an external caller	one protocol, in-process or over a socket	`19 ops · 15 routes`	`control/plane.py:23`
6 · ToolSpec + TaskAgent	a tool author	a spec file, and four phases of adapter	`15 fields · 4 phases`	`tools/spec.py:75`

What is shared, and is the point

- All six reach one executor, one design tree, one append-only provenance log.
- All six are refused the same way: five gates, a dry run, and the trust interlock.

What is enforced, not merely intended

- 10 layers in an allow-list walked by `ast`: `policy` may import only core, and `core` imports nothing internal.
- A new cross-layer import fails the suite by default — slide 29.

mined from the declarations themselves: run_model.py::surfaces() · slides/impress-a-api-user-slides @ bf32d1d

FRAMING · THE SYSTEM AS A MEDIATOR

One side: what the caller states, and what comes back

What “better” means

`goal` and `objectives` — name, direction, min/max. The only statement of success in the system, and what the front is ranked against

What may run

the toolkit inventory, discovered at runtime from a directory: 11 tools in 5 toolkits here. `stages` names which of them this campaign may use

What it may spend, and where

`budget`, `site` caps, `concurrency`, the `replicas` CAP, `backend` + config, three timeouts — of 22 `CampaignSpec` fields

Who decides

`--model` picks one of 6, fixed at launch (ADR 0005) — or your own `ControlPolicy`, which is 3 methods

IMPRESS-A

admits
composes
refuses
dispatches
records

and decides
when to stop

What it tells you while it runs

- `observe()` returns the SAME object the policy's `decide` is handed — one truth, not a dashboard view of it.
- An event stream, cursor-paged or tailed; provenance (14 records across 5 kinds in the run behind this deck); a durable ledger; `artifacts(selector)`; a heartbeat.

What you can say back, mid-flight

- `steer`, `pause`, `resume`, `stop`, `backtrack`, `request_human`, `ingest_measurement` — 19 control-plane operations in all.
- A measurement SUPERSEDES a prediction and never deletes it, which is why the front is not monotonic.

The asymmetry is the whole design: the caller never names a task, a node, or a tool parameter — and the engine never invents an objective or a budget. Intent and constraints go in; tasks, and evidence about them, come out. Everything on the next slide is what the system does with that gap.

FRAMING · THE SYSTEM AS A MEDIATOR

The other side: how that becomes processes on nodes

INTENT	COMPOSE	ADMIT	PARAMETERIZE	DEPLOY	ENGINE	TASKS
6 fields, abstract	*stages → typed nodes*	*5 gates, then trust*	*per node, in order*	*one factory, every node*	*asyncflow + a backend*	*processes, on nodes*
campaign params merge BENEATH the policy's; `replicas` is a cap, not a default	ports are `ArtifactType`, so a mismatch is a compose-time error; P6 tools are inlined, never scheduled	five gates, the interlock's cost cap, and a forced dry run for a shape nobody has reviewed	spec defaults ← campaign params ← policy params, and `frozen_parameters` beat all three	`_run.__name__ = node_id`, because asyncflow names a task from `fn.__name__`	`WorkflowEngine` with a work_dir; the backend resolves BY NAME from site config	4 nodes over 2 lineages in the mock run; on Delta, six stages pinned to GPUs
executor.py:419	composer.py:36	validate.py:30	resources.py:14	dispatch.py:144	backend.py:97	agent.py:120

← and back up the same path: a typed TaskResult (ArtifactRefs, metrics, a QCReport, cost) → to_outcome() → executor._absorb, the single await-free writer → tree, front, provenance, ledger

dispatch.py:91 · the gates run on the way OUT of a node, so a tool that exits 0 and lies is still failed here

The harness nobody sees

- Resource shapes are NOT portable, so translating them is ours: RADICAL takes `ranks`/`gpus_per_rank`, Dragon takes process templates, the concurrent backend takes nothing at all. One `ResourceShape` in the spec, three descriptions out.
- No backend class is named outside `exec/backend.py` — site config picks one by name.

Two asyncflow idioms, not interchangeable

- An UNAWAITED future passed as an argument IS a DAG edge — `dispatch.py:182`, walking `topo_order()`. That is how a stage list becomes a dependency graph with no scheduler of ours.
- `asyncio.gather(..., return_exceptions=True)` awaits INDEPENDENT work — replica fan-out — and one bad task must not cancel its siblings.

What this picture does not show: every realized task in the run behind this deck is a mock, and on the real six-stage chain this path has only ever run one lineage wide.

the engine side of six surfaces — the sibling deck argues this seam, this one cites it

ACT 1 OF 3

FUNCTIONALITY

Not features — tasks. What can you write against this that you cannot write against a pipeline?

A tool is a declaration, not a code path
15 declared fields; loading one VALIDATES it, and a bad toolkit is disabled whole

A policy emits intent, never a DAG
4 decisions, 6 intent fields, and no tool parameter names anywhere in policy/

Composition is free; refusal is cheap
five gates and a dry run before anything executes; a rejection costs one cycle and hands back a reason

The weakest claim in this act: every toolkit that exists is one of the 4 real stages of one chain, plus mocks — and no one outside this package has authored either a toolkit or a policy

Surfaces 1 and 2: the operator's and the scientist's

```
stages: [rfd3_design, ligandmpnn_design, packmin, fastrelax, filter_shape, boltz_predict]
params:
  rfd3_design:      {input_spec_path: data/alr/ALR_binder_design.json, diffusion_steps: 50}
  boltz_predict:    {ligand_smiles: "COc1cc(S(=O)(=O)[O-])c(C)cc1/N=N/..."}
replicas: 4          # N INDEPENDENT lineages, one DesignNode each
objectives:
  - {name: total_score,           direction: minimize}
  - {name: shape_complementarity, direction: maximize, min: 0.55}
  - {name: ligand_iptm,           direction: maximize, min: 0.4}
```

campaigns/delta-small-molecule.yaml:30 · TRIMMED

impress-a	what it does
`run <spec>`	--model A | B | C | D | null | replay; the model is fixed at launch
`tools`	lists the 11 registered tools with pattern and gate count
`preflight [spec]`	checks the real-toolkit environment BEFORE the queue

What is in here, and what deliberately is not

- In: the scientific question — which stages, which ligand, which contig, what counts as better.
- Not in: how to explore. No thresholds, no schedule, no retry policy, no tool invocation.
- A relative `*_path` resolves against the YAML, not the CWD — the launcher cds into a per-job dir.

One objective, two sources

- An objective names a property, not a predictor.
- A surrogate today and a wet-lab assay tomorrow are the same declaration — `authority` decides which wins (decision 0012).

the objectives on this slide are the four the Delta campaign declares; the mock campaign declares its own

ACT 1 · FUNCTIONALITY

Surface 3: a reasoner is three methods and four answers

```python
class ControlPolicy(Protocol):
    name: str

    async def decide(self, obs: CampaignObservation) -> Decision: ...
    async def interpret(self, results: RunOutcome, obs) -> str | None: ...
    async def on_rejected(self, decision, failure: ValidationFailure) ->
Decision: ...
```

policy/base.py:19 · EDITED (annotations trimmed)

```python
Decision = Union[ComposeAndRun, Backtrack, RequestHuman, Stop]
```

core/decision.py:56 · VERBATIM

ComposeAndRun
run this experiment

Backtrack
branch from an earlier node

RequestHuman
stop and ask

Stop
the campaign is done

Two things that are absent on purpose

- `conduct` is NOT a member of the Protocol. Requiring it would break every existing policy, and `runtime_checkable` makes the membership list load-bearing for isinstance.
- There is no `Await` or `Cancel` decision. Waiting is something the reasoner DOES by calling `result`, not something it asks the loop to do — one control-flow vocabulary, not two.

Model	Who decides	Mode
A	four-node LangGraph loop: hypothesize → parameterize	autonomous
B	one structured LLM call over rich campaign state	autonomous
C	an external caller, over the control plane	headless
D	rules, bandit, Bayesian opt, or replay	reproducible

Why a fifth model is cheap

- Subclass `BasePolicy`, implement one method. `interpret` defaults to None and `on_rejected` defaults to Stop — "give up rather than loop".
- Policies compose: `RuleCorrectionsPolicy` wraps any of them as an unconditional guard (the CLI applies it by default — max_replicas 8, `mock_noodle` forbidden), `LoggingPolicy` for provenance.
- The mock run in this deck was `D` under that guard: policy printed itself as "rulecorrectionspolicy(threshold)".

ADR 0005: the control model is fixed at launch · ADR 0007: one observation, two consumers

ACT 1 · FUNCTIONALITY

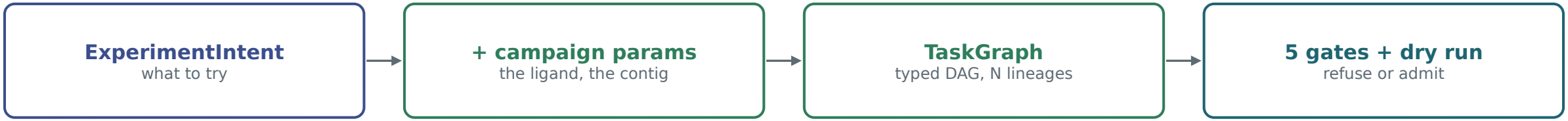
The seam that keeps policies swappable: intent, not a graph

```
class ExperimentIntent(BaseModel):
    goal: str
    toolkit_hint: str | None = None
    stages: list[str] = []        # tool ids, in order
    params: dict[str, Any] = {}
    parent_node: str | None = None
    replicas: int = 1
```

core/decision.py:21 · EDITED (Field(...) elided)

No policy in this package names a tool parameter

- `_apply_campaign_defaults` merges the campaign's `params` BENEATH whatever the policy asked for: the policy always wins on a key it set.
- So a `conduct` reasoner and an external directive get the ligand for free, and no policy has to learn that `boltz_predict` takes `ligand_smiles`.



What this buys

- A policy that cannot name a tool parameter cannot quietly become a pipeline.
- `policy/` may import `core` and nothing else — enforced by `ast`, slide 29.
- Swapping A for D changes the reasoner and nothing below it.

What it costs

- A policy cannot express a graph shape the composer cannot build from an ordered stage list — so free composition is free within one topology family.
- Every graph composed in this deck's run was the same 4-tool chain at two widths.

runtime/executor.py:419 — the merge, and the comment that explains why it lives there

Surface 6: a tool is a file a scientist can review

```
id: rfd3_design
toolkit: rfd3
pattern: P1                       # GPU-node-local, in-job
entry: impress_a.tools.rfd3_agents.RFD3DesignAgent
resources: {gpus: 1, cores: 8, walltime_s: 900}
outputs:
  backbone: {type: Backbone}
parameters:
  diffusion_steps: {type: int, default: 50, min: 20, max: 200,
                    trades: "quality vs wall-clock"}
metrics: [ss_fraction, num_models]
qc_gates:
  - {id: output_present, params: {key: result}}
  - {id: has_secondary_structure, params: {min_ss_fraction: 0.3}}
cost_model: {unit: invocation, cost: {gpu_hours: 0.1}}
```

toolkits/rfd3/tools/rfd3_design/spec.yaml:1 · TRIMMED

Loading is validating

- A default outside its own declared range, a parameter both varyable and frozen, a gate id that is not implemented, a duplicate tool id, a SKILL.md missing one of 4 required sections — all LOAD-time errors.
- A toolkit that fails anywhere registers nothing: all-or-nothing per directory, so an RFD3 container problem cannot also take down PyRosetta.

Read by four consumers

- The composer types the graph from `inputs`/`outputs`; the validator sizes it from `resources`; the budget guard prices it from `cost_model`; the task agent resolves `parameters` and enforces `qc_gates`.
- Declarative so a scientist can review it in a pull request (decision 0011).

```
if self.pattern is Pattern.P1 and self.resources.gpus == 0:
    raise ValueError(f"{self.id}: P1 declared but resources.gpus
== 0")
...
if g.role == "acceptance" and g.id == "metric_in_range" \
        and g.params.get("metric") not in reported:
    raise ValueError("... needs an integrity"
                     " metrics_reported gate naming it")
```

tools/spec.py:100 · EDITED

And the rule behind the second check

- An acceptance threshold must never be the only thing that notices a value is missing — that is how a fabricated 0.0 passes for a measurement.
- So integrity and acceptance are separate roles, and only integrity failures count against a workflow's trust (decision 0013).

11 tools load clean at slides/impress-a-api-user-slides @ bf32d1d; registry errors: 0

ACT 1 · FUNCTIONALITY

Four phases, and the one you may not override

pre_process
TaskRequest in, TaskRequest out

→

parameterize
defaults resolved, frozen refused, seed drawn

→

run
the only phase an adapter really writes

→

post_process
metrics extracted, GATES ENFORCED

__call__()
the four phases, in order: NOT overridable, and pattern dispatch belongs to exec/

```
async def post_process(self, req, raw) -> TaskResult:
    """Extract metrics and ENFORCE QC gates. Never skippable."""
    qc = gates.evaluate(self.spec, raw)
    return TaskResult(tool=self.spec.id,
                      outputs=self._as_artifacts(raw.get("outputs", {})),
                      metrics=raw.get("metrics", {}), qc=qc, ...)
```

tools/agent.py:110 · TRIMMED

Why the type comes from the spec, never the adapter

- `_as_artifacts` stamps each output with the port type the composer type-checked against, so a ref cannot disagree with the contract.
- An adapter returning an undeclared output is an ERROR, not a dropped value — dropping it silently would hide a spec bug.
- A `Path` means a file and gets hashed; anything else is an inline value.

Gate	What it catches
`output_present`	the adapter returned nothing under the declared key
`metrics_reported`	a promised metric is absent, or not a finite number
`metric_in_range`	a value outside the acceptance band
`has_secondary_structure`	the silent "designed a noodle"

Why a zero exit code is never enough

- Tools fail SILENTLY far more often than they crash: an empty output directory, an unrecognised header, a relax that diverged.
- 20 fixtures ship with the toolkits, 11 of them `.bad` — recorded failures kept as tests. A QC-failed node is never rankable.

gates are registered by name with a decorator: tools/gates.py — adding one is a function and a spec entry

ACT 1 · FUNCTIONALITY

Five gates, a dry run, and a refusal from the real run

```
def validate(self, g: TaskGraph) -> ValidationFailure | None:
    for gate in (self._g1_types, self._g2_structure, self._g3_params,
                 self._g4_resources, self._g5_budget):
        if (f := gate(g)) is not None:
            return f
    return None
```

compose/validate.py:30 · VERBATIM

Gate	Refuses
1 · types	a Backbone where a Complex is required, an unknown port
2 · structure	a cycle, an orphan, a P6 the composer should have inlined
3 · params	out of range, frozen, unknown to the spec
4 · resources	more GPUs than the site has; external when not allowed
5 · budget	an estimate the ledger cannot afford
+ dry run	every agent parameterizes, nothing executes

Why free composition is affordable to get wrong

- A rejected graph costs one cycle — nothing has executed yet. The policy gets the gate, the node and the reason, then another attempt, bounded by `max_attempts`.

The field that keeps a policy from degrading itself

- `ValidationFailure.transient` — True when the graph is FINE and only the moment is wrong. Gates 1-5 are properties of the graph and never transient; the interlock's cost cap is.
- Without it, a policy answers contention by mutilating its own intent.

FROM THE RUN THAT BUILT THIS DECK

```
graph 1 · 3 lineages · 12 tasks
REFUSED at the interlock gate, transient=true
"provisional pattern capped at 10% of available cpu_hours (1.500 > 1.200)"
then: 2 graphs admitted at 2 lineages, 8 tasks each
```

What the refusal above is NOT

- Not a bug, and not the interlock being clever: the policy asked for more lineages than an untrusted pattern may spend, and shrank. On the real chain that is backlog A10.

mined: run.json · mock.refusals[0] · 3 graphs composed, 1 refused, 2 admitted

ACT 2 OF 3

PERFORMANCE

What the loop costs, where it serialises, and which of those numbers I am actually entitled to say.

The reasoner is no longer the loop body
10 session operations; submit many, collect in the order they FINISH, not the order you asked

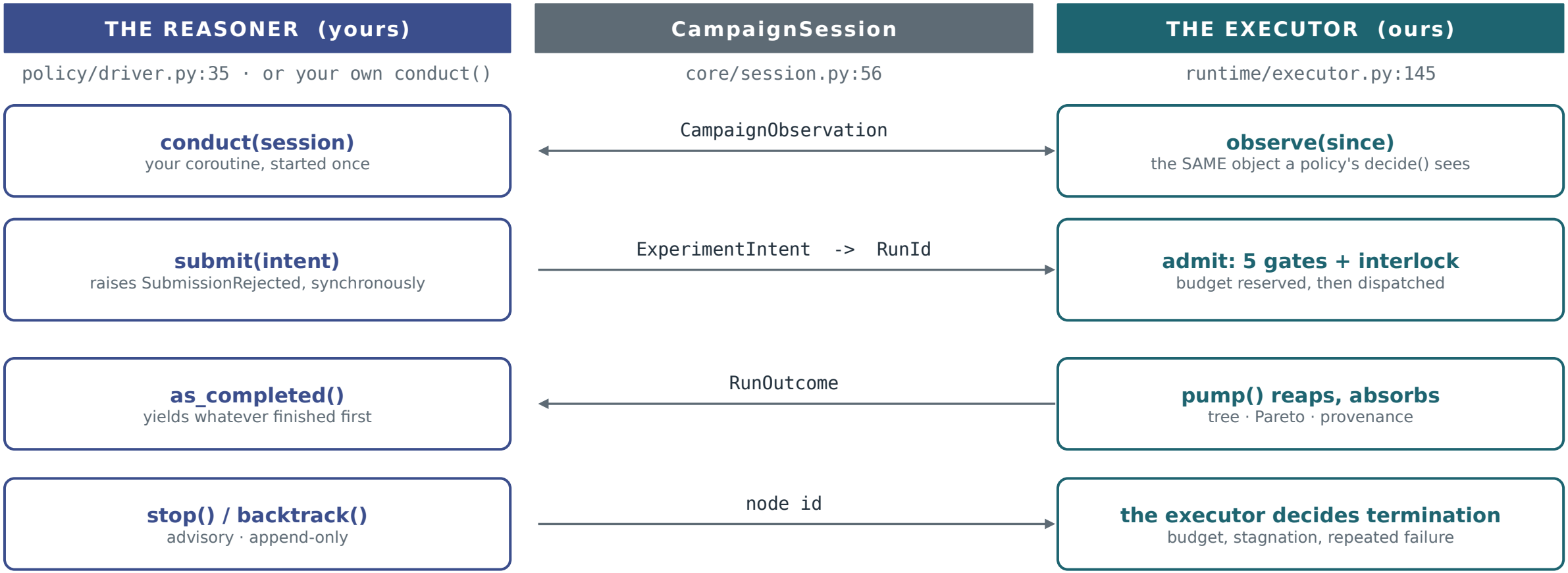
Refusal is measurably cheap
the refused graph and its replacement were composed inside the same millisecond of the run

Steady-state cycle cost is small and measured
cycle 2 cost 69 ms against cycle 1's 1.09 s — the difference is engine construction, once

The weakest claim in this act: there is no concurrent run and no serial baseline anywhere in this deck, so nothing here divides into a speedup — and the stages were mocks, so this times the ENGINE and never the science

ACT 2 · PERFORMANCE

Surface 4: the reasoner calls the campaign



Request/response, and nothing live crosses it

- Every argument and every return value is serializable — which is what lets the reasoner sit on a login node while the executor sits in the allocation.

It lives in `core`, and that is load-bearing

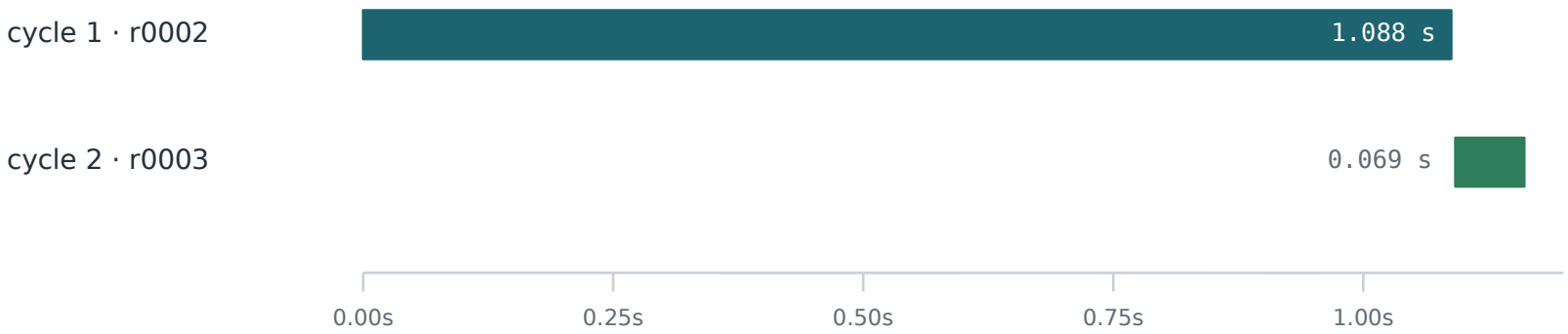
- A policy may import only core (ADR 0010), so anything a reasoner touches has to be typed in core or the seam rots.

there is no Await or Cancel decision: waiting is a call, not a request to the loop

ACT 2 · PERFORMANCE

One campaign, timed by its own provenance

composed → dispatched, per cycle



the whole difference is engine construction, paid once: 1.02 s of it

at	what the campaign recorded
0.000s	rejected r0001 g000001-c783
0.000s	composed r0002 g000002-c783
1.088s	dispatched r0002 g000002-c783
1.091s	run_finished r0002
1.092s	composed r0003 g000003-c783
1.161s	dispatched r0003 g000003-c783
1.162s	run_finished r0003
[illegible]	campaign_ended target_front_size_3

What this measures, and in what units

- The campaign's own provenance clock: first record to `terminated` is 1.17 s. The process took 2.8 s, so roughly 1.6 s is interpreter startup and imports, OUTSIDE the loop.
- Host: one laptop, no allocation. Stages: mocks. 4 nodes absorbed, front of 3, stop reason "target front size 3 reached".

The irregular detail is the useful one

- Cycle 1's 1.09 s includes building the execution engine. Cycle 2 reuses it and costs 69 ms for compose, validate, dispatch, absorb and rank — with four mock tasks across two lineages in it.

Denominators, stated

- 1 host · 1 experiment in flight · 2 lineages per experiment · 4 mock tasks per lineage.
- Nothing was varied. No pool width, no replica width, no node count, no real tool.
- So: a latency figure for the engine, and NOT a throughput or scaling result.

mined: run.json · mock.timeline · slides/impress-a-api-user-slides @ bf32d1d · one laptop, mock stages, no allocation

ACT 2 · PERFORMANCE

What I measured, and what I therefore do not claim

Measured	Value	From
campaign span	1.17 s	provenance ts
steady-state cycle	69 ms	timeline
refuse + recompose	< 1 ms	timeline
provenance written	14 records	5 jsonl files
ledger records	7	jobs/ledger.jsonl
declared cost of a run	0.11 gpu-h	cost_model
tests	168	pytest --collect-only

Where this stops being a systems question

Four lineages do not buy four times the chance of a good design.
The acceptance rate across the project's real runs is about 31%, and nothing in this engine improves that — it only decides how many draws you can afford and whether you can tell a bad draw from a broken tool.

So the figure that matters is not tasks per hour. It is how few GPU-hours it takes to learn that a workflow shape is not worth running again — which is what the trust interlock is for. Next slide: what the two levers are actually worth, measured next door and never here.

Measured, and in this deck

- Engine latency per cycle, with engine construction separated out.
- The cost of a refusal — it is a composition, not a run.
- That the provenance log is complete enough to time the campaign from outside it.
- That 2 lineages absorb as 2 distinct nodes with distinct metrics.

Not measured — so not claimed

- No concurrent run: `concurrency > 1` never exercised here, so the one thing the session API exists for is unmeasured.
- No serial baseline, so nothing above divides into a speedup.
- No real tool: every stage was a mock, so there is no science-per-GPU-hour figure of any kind.
- No multi-node, no backend comparison: `concurrent` only. Dragon and radical are selected by name and untouched here.
- On the real chain, `replicas > 1` has never run at all (backlog A10).
- Neither lever that moves the science — packing, or the decision layer — has been measured on this engine. Both are measured next door, by other code, next slide.

the 31% acceptance figure is from the project's Delta run archive (backlog A1), not from this checkout

ACT 2 · PERFORMANCE · MEASURED NEXT DOOR, BY OTHER CODE

The two levers that move the science — and nobody has pulled both

one policy, held fixed	k=1	k=2	k=4	k=6	k=8
backbones / node-hour	57.8	105.6	179.7	258.2	319.1
passing folds / GPU-hour	3.96	6.30	9.95	12.06	20.55
high-quality folds / GPU-hour	2.13	2.40	4.83	5.43	10.88

PACKING, decision layer fixed: ×5.52 backbones per node-hour, ×5.11 high-quality folds per GPU-hour, from k=1 to k=8.

four decision layers, one hour, k=1	pass/GPU-h	HQ/GPU-h	pLDDT
scripted route · 23110801	12.12	5.77	88.2
Beam (reflection) · 23225651	8.08	6.35	94.4
Qwen2.5-72B, in-allocation · 23207314 (2 nodes)	6.35	3.17	92.2
claude-sonnet-5-5 pilot · 23112484	5.77	3.46	86.2
Qwen3.5-122B, in-allocation · 23229812 (2 nodes)	2.88	0.58	83.8

The arithmetic

- Packing: about ×5.52 at a fixed policy.
- Decision layer: about ×3 at matched packing — the best agentic arm reaches 6.35 HQ/GPU-h at k=1, where this same router reaches 2.13 at k=1 and 10.88 at k=8.
- No run has ever had both.

What one hour cannot say

- 12–25 folds per arm in the window, so the order WITHIN the agentic group is not a ranking.

“Agentic” is not a property that predicts anything. Qwen3.5-122B reaches 2.88 passing folds per GPU-hour where the scripted route — which has no model in it at all — reaches 12.12. The spread across four models (2.88 to 8.08, pLDDT 83.8 to 94.4) is larger than the gap between “model” and “no model”.

And none of this ran through IMPRESS-A. This engine has measured neither lever: `replicas` IS the packing lever and has never exceeded 1 on the real chain, and every stage it has ever timed was a mock.

TRANSCRIBED, from impress-agentic-performance report.md §§10-12 (verified, 19 checks) · single runs, one hour each, one policy, one storage partition, and k=8 is the largest rung anybody ran rather than a measured maximum

ACT 2 · PERFORMANCE

Where the API refuses breadth — and a seam in it

A novel workflow shape is neither blocked nor trusted on sight

it runs under a cost cap and a forced dry run, is marked suspect whatever the outcome, and promotes only after promote_after = 3 runs with no INTEGRITY failure — demoting on one

Pattern signature	composed at	runs	trust events
`9fdcfd202951…`	3 lineages	1	seen, then refused
`66bb492308e3…`	2 lineages	2	seen, 2 clean

Two widths of one chain are two patterns — not what the docstring says

- `pattern_signature` is documented as SHAPE ONLY: tool ids plus typed edges, with parameter values excluded deliberately so a parameter change cannot reset trust.
- But it is built per NODE, so a 3-lineage graph contributes 3 copies of every hop and hashes differently from the 2-lineage one. Replica width is in the signature by construction, not by decision.

```
parts = sorted(
    f"{n.tool}<-{','.join(sorted(self.nodes[d].tool for d in n.deps))}"
    for n in self.nodes.values()          # <- one entry PER NODE
)
return hashlib.sha256("|".join(parts).encode()).hexdigest()[:16]
```

compose/graph.py:51 · VERBATIM

Why it bites

- Trust earned at width 2 does not transfer to width 3.
- 2 clean runs here, against promote_after = 3 — so 0 promotions, and all 4 nodes came out `suspect`.
- To run the real 4-lineage independence check you must first earn trust AT 4.
- Backlog A10 names this; it is a known cost, not a discovery.

mined: run.json · mock.widths_per_signature · trust: 2 seen, 2 clean, 0 promoted

ACT 3 OF 3

USABILITY

Not the interface — the use cases. Is this an application someone opens, a platform others plug into, or a component something else drives?

As an application: strong

one YAML, one command, 3 subcommands — and a preflight that respects what an allocation costs

As a platform: arguable

11 tools discovered at runtime from a directory; $IMPRESS_A_TOOLKITS adds yours without touching the package

As a component: empty

19 control-plane operations and a RemoteSession that implements the reasoner's own protocol — and no caller

The weakest claim in this act: nothing outside this package has ever driven a campaign — the four-hour live run three slides from now is the reference pipeline's toolkit with an LLM on it, not this engine — and `--model C` with no caller falls back to model D after 5 s

ACT 3 · USABILITY

What running one is actually like

1

You write the file

22 keys exist. A real campaign uses about twenty lines: the goal, the budget, the objectives, and the stages you trust

2

You run preflight

On the login node, before the queue, where repairing anything is free. ok / FAIL / undetermined — and undetermined never reads as a pass

3

You stop watching

There is no step list to babysit. It observes, decides, composes a graph, type-checks it, gates it, and dispatches

4

It argues with itself

It asked for 3 lineages. The interlock refused on cost. It came back with 2 and was admitted — nobody woken up

5

It stops, and says why

"target front size 3 reached". 3 designs on the front, 4 nodes, every one marked suspect until a human has looked

```
$ impress-a run campaigns/delta-small-molecule.yaml --model D
```

What you never write

- No DAG, no step list, no retry loop, and no per-experiment edit to a batch script — the shape of the work is composed each cycle from the stages you named.

What comes back: one directory per job — manifest.json (campaign, model, git sha and whether the checkout was dirty), campaign.log, provenance/*.jsonl (14 records across 5 kinds), jobs/ledger.jsonl (7)

you can hand the directory to someone else and they can reconstruct every decision, including the one that was refused

The experience it is trying to buy: you spend your attention on what you want, not on what order to ask for it in — and the first thing you read when it finishes is the reason it stopped.

ACT 3 · USABILITY

When it says no, it says what to change

Before the queue

preflight, on the login node

`$MPNN_DIR has no model_params/ — run its get_model_params.sh`. Seconds to fix, no allocation spent. A timeout reports as undetermined, never as a pass.

Before anything runs

five gates, then the interlock

"provisional pattern capped at 10% of available cpu_hours (1.500 > 1.200)" — marked transient, so the policy shrinks the request and tries again instead of stopping. Cost of the refusal: one composition, no GPU.

After a tool succeeds

QC, on the way out of the node

A tool can exit 0, report 0.91 designability, and have produced a noodle. The integrity gate fails the node anyway, and a failed node never reaches the front.

The one place it does NOT say no loudly — and it is the one that cost a real run

- Cost-model figures sat 6–69× over measurement, so gate 5 summed past the cap and the policy quietly truncated `boltz_predict` off the end of the chain. The campaign ran. It just could not produce the objective it was launched for.
- It warns now — `_objectives_without_a_producer` — and a warning is not a gate. If you take one thing from this slide: a refusal you can read is a feature, and the dangerous refusals are the ones phrased as a smaller plan.

```
interlock  provisional pattern capped at 10% of available cpu_hours (1.500 > 1.200)
           transient=true  →  policy recomposed at 2 lineages, admitted
```

runtime/executor.py:391 · run.json mock.refusals[0] · MINED

the dangerous refusal is the one phrased as a smaller plan

ACT 3 · USABILITY · EVIDENCE FROM OUTSIDE THIS PACKAGE

Next door, a model drove the real toolkit for four hours

4 h, 1 node, 4 pipelines, same inputs	state machine	an LLM, calling 8 tools
who decided what ran next	`adaptive_decision()`	`claude-sonnet-5-5`
rfd3 per pipeline-hour	15.5	16.2
distinct passing designs	53	176
GPU-hours per passing design	0.302	0.083
backbones that yielded a fold	21%	74%
fastrelax pass rate	49%	88%
mpnn tasks spent	858	349
model cost	–	$25.40 ($0.14/design)

Two caveats, load-bearing — and both got WORSE with the 2026-10-26 analysis. The weak baseline is now MEASURED, not suspected: with the decision layer held fixed at adaptive_decision(), the reference pipeline's own fastrelax pass rate fell 63.6% -> 49.4% between 22701168 and this run, and only 1.3 of those 14.2 points is explained by binder length. Do not quote the 3.3x. And the fastrelax gap is no longer the decision layer's to claim: the agentic HARNESS pack-minimises before relaxing and the reference harness never does (0 of 5,002 decoys across 14 runs) - including the scripted route at 48 of 48, which has no model in it. It is a confound between the two examples.

What it did unprompted

- 3 of 4 pipelines called finish deliberately, with an accurate account of what they had produced.
- And one pipeline diagnosed the fa_rep relax bottleneck unprompted — a bottleneck that was already an open backlog item next door.

And what it did badly

- score_interface called 13x against 27 relaxes - the cheap CPU gate before an expensive fold is being skipped
- inspect_ensemble called once in 137 turns

Both arms ran the full four hours with zero task failures. Throughput is equal, so the round trip costs nothing measurable — the difference is entirely in which work the funnel chose to do, and most of THAT is now the harness rather than the model. This is not IMPRESS-A: no campaign engine, no gates, no interlock, no trust.

TRANSCRIBED · examples/agentic_small_molecule_binding/CLAUDE.md - transcribed from its revision history; the run outputs live under $WORK_DIR on Delta, not here · the tool list is mined, from examples/agentic_small_molecule_binding/tools.py:35

ACT 3 · USABILITY

What those four hours say about these surfaces

What the live runs showed	Which surface answers it — and whether it has been exercised
The round trip is free: 16.2 against 15.5 rfd3 per pipeline-hour	A reasoner per experiment is affordable, which is the bet surfaces 3 and 4 are built on. Here it is measured against MOCKS only.
It skipped the cheap gate before expensive folds, and never looked at its own ensemble	The engine does not rely on the reasoner's discipline: gate 5 and the interlock refuse on cost BEFORE anything runs, and a policy cannot name a tool parameter at all.
A work dir the container could not see read exactly like a model failure	Exactly the class `impress-a preflight` exists for — and it would NOT have caught this one: it checks the login node, not what a container binds. An honest gap.
The model can live inside the allocation, and it is NOT free: a second billed node plus 27% of the allocation before the first task event (0.243 GPU-h/passing fold, not 0.121)	That is what model C's endpoint could be, and the startup tax argues for LONG campaigns rather than short ones. Today `--model C` with no caller falls back to model D after 5 s rather than failing — ask 5.

Say this out loud: none of that ran through this package. Different code, no campaign engine, no gates, no trust ledger, one pipeline per GPU. It is evidence that the idea works on this hardware, and it is not evidence that this engine does. The component claim is still empty, and the nearest thing to a user it has is still me.

the live figures are TRANSCRIBED · examples/agentic_small_molecule_binding/CLAUDE.md - transcribed from its revision history; the run outputs live under $WORK_DIR on Delta, not here

ACT 3 · USABILITY · THREE RUNS KEPT AS EVIDENCE

How an agentic run fails on a cluster — and what here would catch it

The failure	How it looked	What in IMPRESS-A would catch it
the silent fallback · 23219379	Four pipelines fell back to the scripted route on turn two and ran the hour without a model. 46 relaxes, 35 passing folds, a transcript that reads entirely healthy.	NOTHING. Gates check artifacts, not who chose them; a scripted route emits well-formed work that passes every one. There is no degraded flag on a Decision and none in the ledger — ask 5.
writes that vanished · 23206285	Every rfd3 write landed in the container's tmpfs overlay and vanished. rfd3 logged success, out/ was empty, and the model correctly concluded its backbones failed.	Gate 1 (count_matches_request) catches the empty result — as a SCIENCE failure. The interlock then counts it against the pattern's trust, so a broken bind demotes a good workflow shape.
the run that cost nine seconds · 23231485	The launcher's pre-flight found the model's API key missing in the submitting environment and aborted in 9 s, before claiming the GPUs. No run tree at all.	`impress-a preflight` — this is exactly the class it exists for, and the only one of the three it covers. Tri-state, and a timeout is never a pass.

One of three, and it is the cheap one. of the three, only the third was cheap and only the second was loud. A harness that degrades into a WORKING fallback produces plausible output at full rate, and nothing but an explicit flag distinguishes it. The two detectors that separate it are both outside this package: a degraded-turn count, and prompt tokens repeating the previous turn on 98.6% of turns against 0–5.9% on every healthy arm. **IMPRESS-A records which policy ran and every artifact it produced, and nowhere records that the policy stopped being the policy.**

the three runs are TRANSCRIBED, from impress-agentic-performance report.md §14 · the degraded path and the work-dir guard are MINED, from examples/agentic_small_molecule_binding/agentic_pipeline.py:136 and examples/agentic_small_molecule_binding/delta_gpu_run.sh:233

ACT 3 · USABILITY

As an application: one command, one honest gate

```
$ impress-a preflight campaigns/delta-small-molecule-smoke.yaml

  preflight

    [ok  ] $WORK_DIR            /work/nvme/<project>/<user>
    [FAIL] $MPNN_DIR            no model_params/ - run its get_model_params.sh
    [ok  ] boltz CCD cache      extraction proven complete
    [warn] pyrosetta            timed out after 120s - a loaded login node can
                                take that long to import it; re-run
    [ok  ] venv on fast storage /work/nvme/<project>/<user>/venv

  7/9 ok, 1 undetermined (pyrosetta) - missing: $MPNN_DIR
```

cli/__init__.py:252 · ILLUSTRATIVE (shape of real output)

Why the third state exists

- Every real adapter raises on a missing env var — but only once a task is RUNNING, which on HPC means inside the allocation, after the queue wait.
- `ok` is tri-state: None means UNDETERMINED, not failed. A probe that shells out to import a heavy science stack can take minutes on a busy login node, and reporting that as FAIL trains the reader to ignore the FAIL column.

The rule that must not erode

- None is ONLY for a probe that produced no answer. A deterministic negative is always False: a missing `$MPNN_DIR` is a FAIL forever; a pyrosetta *import error* is a FAIL, a pyrosetta *timeout* is undetermined.

What it caught that nothing else would

- A venv on HDD rather than NVMe — which works perfectly, about 8x slower, and lands as an unexplained task timeout inside an allocation.
- `ml_collections`, which only LigandMPNN's bundled openfold imports.

Each job writes one self-contained directory: manifest.json (job id, campaign, model, git sha + dirty), slurm.out, campaign.log, provenance/*.jsonl, jobs/ledger.jsonl

a run you can hand to someone else, including whether the checkout was dirty when it started

the output above is the shape of real preflight output, with paths replaced by placeholders — see CODE_FOR_DECK.md

ACT 3 · USABILITY

Surface 5: one protocol, two transports

Route	Semantics that are NOT the transport's invention
`POST /campaigns/<c>/runs`	202 with a run id, or 409 with the ValidationFailure that refused it
`GET …/runs/<r>/result?wait=`	200 outcome · 204 ask again · 410 the campaign ended
`GET …/observe?since=`	the same CampaignObservation a policy's decide() receives
`GET …/events?stream=1`	cursor-paged page, or a live tail of the same sequence
`POST …/measurements`	a P8 arrival, accepted even after termination

Why admission is synchronous and waiting is a long poll

- Accepting everything and reporting rejections later on the event stream would sever a rejection from the request that caused it, leaving nothing to bound retries against.
- And a socket cannot be held open for a twelve-hour folding job, so the deadline is the transport's problem (MAX_WAIT_S = 60) and re-issuing is the client's — neither is visible to the reasoner, which just calls result() and blocks.

The rule that keeps two transports from becoming two systems

- No adapter may add an operation absent from the core protocol (ADR 0007). 19 operations, 15 route heads, and the routes are a mechanical rendering of the Protocol — which is why run_model.py counts both and prints them side by side.

RemoteSession
implements the reasoner's OWN protocol over the plane — so SequentialPolicyDriver runs unmodified against a socket

Built on asyncio.start_server
no framework. A small fixed dialect, on loopback, for one trusted client

NOT public-facing
no auth, no TLS, one body-size limit. The module says so in its own docstring; bind it to localhost or a private interface only

control/plane.py:23 is the protocol · control/http.py:183 is the whole route table · control/session.py:48 is RemoteSession

ACT 3 · USABILITY

Why the surfaces stay separate: an allow-list in a test

core	**imports NOTHING internal**	✗ **everything**
policy	may import core	✗ **tools · exec · runtime**
compose	may import core, tools	✗ **exec**
tools	may import core, exec	
exec	may import compose, core, tools	
runtime	may import compose, core, exec, policy, tools	
manager	may import core, policy, runtime, tools	
control	may import core, manager, runtime	
cli	may import compose, core, manager, policy, tools	
__main__	may import cli	

The three that are load-bearing

- `policy` → `tools`/`exec` would let a policy reach a tool adapter, and it would stop emitting abstract intent.
- `compose` → `exec` would end the laptop test tier: validation is testable with no backend only because that edge does not exist.
- `core` → anything would make the serializable seam unenforceable.

```
ALLOWED = {
    "core":   frozenset(),
    "policy": frozenset({"core"}),
    "compose": frozenset({"tools", "core"}),
    ...
}
```

tests/test_layering.py:43 · TRIMMED

Why `ast`, not a scan of imports

- The real adapters defer their science imports into `run()` bodies on purpose — so a deferred `from ..exec import …` inside a compose function would be exactly as fatal as one at the top of the file, and `ast.walk` sees both.
- An allow-list, not a deny-list: a NEW cross-layer import fails by default.

10 layers, 168 tests, 9 test files · the table mirrors docs/reference/architecture.md

WHERE THIS ACTUALLY IS

Status, by dimension

FUNCTIONALITY	PERFORMANCE	USABILITY
RUNS	**RUNS**	**RUNS**
Six surfaces, all exercised by the test tier and one real campaign	**Engine latency measured** 69 ms steady-state cycle, mocks, one host	**One YAML, one command** and a preflight that runs before the queue
Load-is-validating 11 tools, 20 gate fixtures, 0 registry errors	**Both ends of the backend bounded** startup and shutdown timeouts, from real hangs	**Toolkits discovered at runtime** $IMPRESS_A_TOOLKITS, no package change
Refuse-and-retry works observed in the run: interlock → shrink → admit	**Provenance is complete** the campaign can be timed from outside it	**Two transports, one protocol** in-process and HTTP, tested against each other
DOES NOT	**DOES NOT**	**DOES NOT**
One topology family every graph is an ordered stage list	**No concurrency measurement** `concurrency > 1` never exercised	**Nothing external drives it** the component claim is empty
No third-party author no toolkit or policy written outside this package	**No baseline, and neither lever** packing ×5.52 and policy ×3 are measured next door	**Fallbacks are invisible** no caller → D after 5 s, and nothing records it
Calibration untested no measurement has ever superseded a prediction	**Real chain at width 1 only** `replicas > 1` has never run on it (A10)	**No serve/reason command** model C is reachable only by embedding

Three green and three red per dimension, deliberately the same length. The red column is not a to-do list — it is what you should not believe from this deck.

if you cut one slide from this deck, do not make it this one

SIX QUESTIONS I ACTUALLY WANT ANSWERED

Asks

COSTS A WRONG ANSWER, OR REAL TIME

1 Should `pattern_signature` normalise replica width?

FUNCTIONALITY

It is documented as shape-only and excludes parameter values on purpose, but emits one entry per node, so width is in the hash. Fixing it makes trust transfer across widths — and weakens the claim that a wide run is a new shape. compose/graph.py:51

3 Should `on_rejected` shrink-on-transient live in the base class?

FUNCTIONALITY

Today the default is Stop and each policy re-implements shrinking. That is where a 4-lineage check quietly becomes a 3-lineage one, with nothing in the log but a correction warning. policy/base.py:54

5 Should a fallback be a first-class flag, not a log line?

USABILITY

`--model C` with no caller falls back to D after 5 s, so a headless campaign silently becomes an explicit-policy one. Next door the same shape ran 284 of 288 turns with no model and looked healthy. cli/__init__.py:103

COSTS WHOEVER ADOPTS THIS NEXT

2 Should an acceptance-failure streak demote a pattern?

PERFORMANCE

Trust counts integrity failures only (decision 0013), so a tool emitting well-formed, low-scored garbage promotes after 3 runs. Right rule, but is the gap real? compose/interlock.py:112

4 Is loopback-only the permanent answer for the control plane?

USABILITY

No auth, no TLS, by design and documented. Before anything external drives a campaign, does this need a token, or does it stay in-allocation forever? control/http.py:26

6 Should model A's imports be lazy?

PERFORMANCE

The CLI imports every policy at module scope, so `impress-a tools` pays for langgraph. About 1.6 s of this deck's 2.8 s run was startup, outside the loop. cli/__init__.py:31

every ask names a file and a line · slides/impress-a-api-user-slides @ bf32d1d

Backup: the compute patterns

Pattern	Meaning	Scheduling implication	Used?
P1	GPU-node-local, in-job	one GPU slot, in the allocation we hold	yes · 7 tools
P2	CPU-parallel fan-out, in-job	a core budget, sized from sched_getaffinity	yes · 4 tools
P3	MPI multi-node, in-job	ranks and nodes, declared up front	no
P4	external HPC job	a durable ledger entry — it outlives the agent	no
P5	network service over HTTPS	allowed only if site.allow_external	no
P6	in-process	MUST NEVER be scheduled — the composer inlines it	no
P7	composite pipeline	decomposed before validation	no
P8	external experiment (robotic lab)	forward-declared; a measurement may arrive after termination	no

The taxonomy is organised by scheduling implication, not by science

- Dispatch depends on the pattern, never on what a tool computes — which is why P6 being scheduled is a composer bug rather than a slow path, and why P4 and P8 are the two that need a durable ledger entry.

mined: run.json · surfaces.patterns · 8 declared, 2 used

BACKUP · NOT IN ANY RUNNING ORDER

Backup: the artifact type lattice

Backbone	**Complex**	**Ensemble**	**ProteinSequence**
SequenceSet	**SmallMolecule**	**Parameterized**	**MSA**
StructuralHit	**Metric**	**MetricSet**	**Trajectory**

```
def unifies_with(self, other: "ArtifactType") -> bool:
    if self is other: return True
    # A Complex satisfies a Backbone requirement (it contains one); not the reverse.
    if self is ArtifactType.COMPLEX and other is ArtifactType.BACKBONE: return True
    if self is ArtifactType.METRIC_SET and other is ArtifactType.METRIC: return True
    return False
```

core/types.py · EDITED (one-lined)

Two distinctions carry real weight

- SMALL_MOLECULE vs PARAMETERIZED — an unparameterised ligand reaching a force field is a type error at COMPOSITION time, not silent corruption.
- BACKBONE vs COMPLEX — so an interface scorer cannot be handed a monomer.

Why scientific types rather than file formats

- 12 types, and every one of them would be ".pdb" or ".json" in a format-based lattice — at which point gate 1 can only check that a file exists, and the composer cannot refuse a graph that is scientifically meaningless but structurally fine.
- This is what makes runtime graph composition checkable at all (Part B doc 05, gate 1).

core/types.py · the lattice is deliberately small: it only needs the distinctions that cause real corruption